

MULESOFT – AWS INTEGRATION PROBLEM SET ·
VOLUME 3 OF 5

100 MuleSoft–AWS Integration Problems

Volume 3 — Serverless & Data: Lambda, DynamoDB, Kinesis & Step Functions. 20 fully worked, real-project problems: invoking compute, reading and writing structured data, streaming in real time, and orchestrating multi-step workflows from MuleSoft.

Problems 41–60

Anypoint Studio

Postman

MuleSoft Developer Edition

Volume 3 — Serverless & Data

Volumes 1 and 2 moved files and messages. Volume 3 moves into compute and structured data: invoking AWS Lambda functions for business logic MuleSoft doesn't own, reading and writing DynamoDB as a fast key-value store, consuming and publishing Kinesis Data Streams for real-time data, and orchestrating multi-step workflows with Step Functions. These are the connectors a MuleSoft integration reaches for once "move this data" turns into "trigger this computation" or "coordinate this multi-step process."

SECTION 1 Lambda & DynamoDB Fundamentals (Problems 41–50)

#41

Invoking an AWS Lambda Function Synchronously from a Mule Flow

AWS

PROBLEM STATEMENT

A tax-calculation Lambda function, already written and maintained by the finance engineering team, needs to be called from a Mule checkout flow to compute sales tax for an order — with the result available before the flow continues.

DETAILED EXPLANATION

Not every piece of business logic belongs inside a Mule flow — a tax calculation with jurisdiction-specific rules that change frequently is often better owned by a dedicated Lambda function maintained by the team that understands tax law, deployed and versioned independently of the integration layer. A synchronous ("RequestResponse") Lambda invocation is the right fit whenever the flow genuinely needs the result before it can proceed, as opposed to fire-and-forget background work, which calls for the asynchronous invocation type covered in Problem #42.

CONNECTORS & COMPONENTS USED

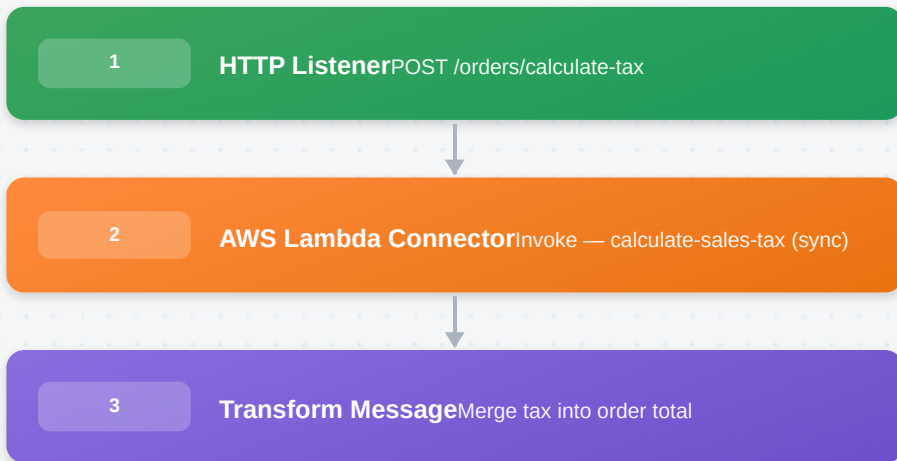
CORE HTTP Listener

AWS AWS Lambda Connector – Invoke

STEPS TO CREATE THE MULE FLOW

- 1 Accept the order subtotal and shipping address over HTTP.
- 2 Invoke the tax-calculation Lambda synchronously with that payload.
- 3 Parse the Lambda's JSON response and add the tax amount to the order total.

CALCULATE-TAX-FLOW



MULE CONFIGURATION XML

```

<flow name="calculate-tax-flow">
  <http:listener path="/orders/calculate-tax" config-
ref="HTTP_Listener_config"/>
  <lambda:invoke config-ref="Lambda_Config" doc:name="Invoke tax
calculator"
    functionName="calculate-sales-tax"
  invocationType="RequestResponse" target="taxResult">
    <lambda:payload>#[%dw 2.0
output application/json
---
{
  subtotal: payload.subtotal,
  state: payload.shippingAddress.state,
  zip: payload.shippingAddress.zip
}]</lambda:payload>
  </lambda:invoke>
  <ee:transform doc:name="Merge tax into total">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
var tax = read(vars.taxResult, "application/json")
---
{
  subtotal: payload.subtotal,
  taxAmount: tax.taxAmount,
  total: payload.subtotal + tax.taxAmount
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

invocationType="RequestResponse"

This is Lambda's synchronous invocation type — the connector call blocks until the function returns, and its return value is available in this same flow, which is what

makes it possible to merge the tax amount into the order total in the very next step.

```
read(vars.taxResult, "application/json") inside the merge transform
```

A Lambda function's return value arrives back through the connector as a raw JSON string, so it has to be explicitly parsed with `read()` before its fields — like `taxAmount` — can be referenced directly.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an `HTTP Listener` onto the canvas and set `Path` to `/orders/calculate-tax`.
- 2 Search the Mule Palette for `AWS Lambda`, drag `Invoke`, and set `Function Name` and `Invocation Type` to `RequestResponse`.
- 3 Write the `Payload` DataWeave script and set `Target Variable` to `taxResult`.
- 4 Drag a `Transform Message` to parse the result and merge it into the response, then save and run.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/calculate-tax

Header	Value
Content-Type	application/json

SAMPLE REQUEST BODY



JSON

```
{ "subtotal": 199.00, "shippingAddress": { "state": "CA", "zip": "94105" } }
```

SAMPLE RESPONSE



JSON

```
{ "subtotal": 199.00, "taxAmount": 17.41, "total": 216.41 }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts the order's subtotal and shipping address, the Lambda Connector's Invoke operation calls the finance team's tax-calculation function synchronously and waits for its result, and a transform parses that result and folds it into the final order total — letting a Mule flow lean on business logic it doesn't own without ever duplicating that logic itself.

#42

Invoking an AWS Lambda Function Asynchronously and Continuing Processing

AWS

PROBLEM STATEMENT

After an order ships, a Lambda function generates a PDF invoice and emails it to the customer — work that takes several seconds and has no bearing on whether the shipping confirmation flow itself succeeds.

DETAILED EXPLANATION

Waiting synchronously for the invoice-generation Lambda would tie the shipping flow's response time to PDF rendering and email delivery — work the caller doesn't need to wait on at all. Lambda's Event invocation type fires the function and returns immediately, without waiting for it to finish or even knowing whether it succeeded; that's an acceptable tradeoff here specifically because invoice generation is not on the critical path of the shipping confirmation itself.

CONNECTORS & COMPONENTS USED

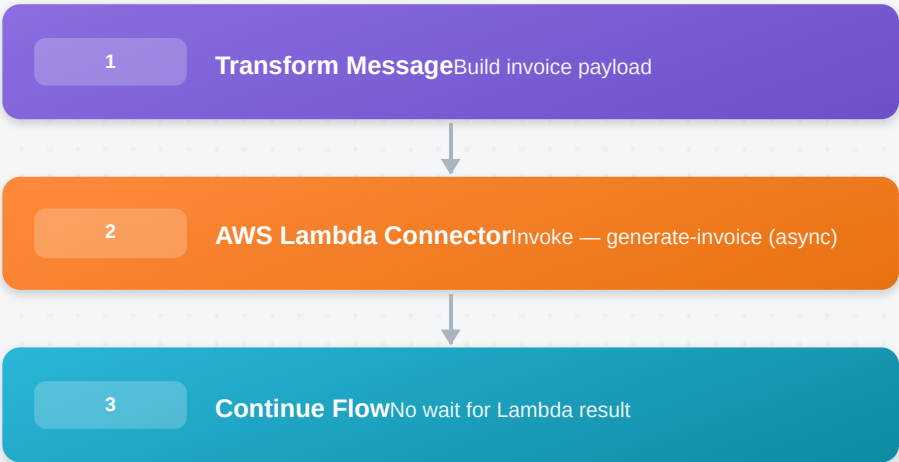
AWS

AWS Lambda Connector – Invoke (Event)

STEPS TO CREATE THE MULE FLOW

- 1 After confirming shipment, build the invoice-generation payload.
- 2 Invoke the invoice Lambda with invocationType set to Event.
- 3 Continue the shipping confirmation flow without waiting for a result.

TRIGGER-INVOICE-GENERATION-FLOW



MULE CONFIGURATION XML

```

<flow name="trigger-invoice-generation-flow">
  <lambda:invoke config-ref="Lambda_Config" doc:name="Fire-and-forget invoice generation"
    functionName="generate-invoice" invocationType="Event">
    <lambda:payload>#[%dw 2.0
output application/json
---
{
  orderId: vars.orderId,
  customerEmail: payload.customerEmail,
  amount: payload.total
}]</lambda:payload>
  </lambda:invoke>
  <logger level="INFO" message="#['Invoice generation triggered for order ' ++ vars.orderId]"/>
</flow>

```

EXPLANATION OF THE CODE

`invocationType="Event"`

This is Lambda's asynchronous invocation type — AWS queues the invocation internally and returns a 202 Accepted to Mule immediately, well before the function itself has actually run, which is exactly the fire-and-forget behavior this problem calls for.

`no target variable capturing a Lambda return value`

An Event invocation's immediate response carries no useful return value from the function itself, so unlike Problem #41's synchronous call, there's nothing meaningful to parse or bind to a variable here.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the AWS Lambda `Invoke` operation onto the canvas after the shipping confirmation logic.
- 2 Set `Function Name` to `generate-invoice` and `Invocation Type` to `Event`.

- 3 Write the `Payload` DataWeave script, then drag a `Logger` to confirm the trigger fired.
- 4 Save and run, then confirm in CloudWatch Logs that the Lambda executed moments later, asynchronously.

TESTING IN POSTMAN

Chained from Problem #33/#40's shipping flow. Trigger a shipment and observe the immediate return with the Lambda executing in the background.

POST

`https://localhost:8081/orders/{orderId}/ship`

CONSOLE OUTPUT



LOG

```
INFO Invoice generation triggered for order 8f2a1c44-...
```

BRIEFING ALL THE STEPS IN WORDS

A transform builds the invoice payload and the Lambda Connector's Invoke operation fires the invoice-generation function with `invocationType` set to `Event`, returning immediately without waiting for the function to actually run — letting the shipping confirmation flow move on instantly, since PDF generation and email delivery have no bearing on whether the confirmation itself succeeded.

#43

Passing a Complex JSON Payload to Lambda and Parsing Its Response

AWS

PROBLEM STATEMENT

A pricing-recommendation Lambda expects a nested payload — a product, its full purchase history, and current market conditions — and returns a structured recommendation with multiple pricing tiers that the Mule flow needs to unpack completely.

DETAILED EXPLANATION

Lambda invocations aren't limited to flat key-value payloads; DataWeave can construct arbitrarily nested JSON on the way in, and just as easily navigate arbitrarily nested JSON on the way back out. The discipline that matters here is keeping the shape of what's sent in sync with what the Lambda function actually expects — since there's no schema enforcement between a Mule flow and a Lambda function the way there might be with a typed API contract, a mismatched field name simply arrives as `null` on the Lambda side with no error raised anywhere.

CONNECTORS & COMPONENTS USED

AWS

AWS Lambda Connector – Invoke

DW

Transform Message

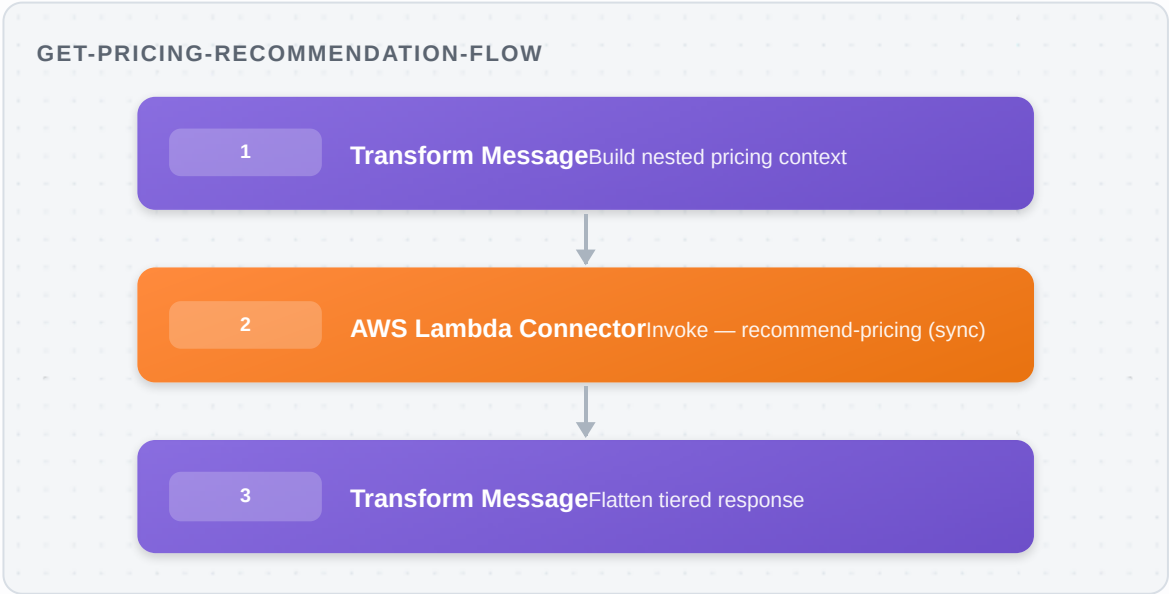
STEPS TO CREATE THE MULE FLOW

- 1

Assemble the nested pricing-context payload from several upstream data sources.
- 2

Invoke the pricing-recommendation Lambda synchronously.
- 3

Parse the nested tiered-pricing response into a flat structure the caller can use directly.



MULE CONFIGURATION XML

```

<flow name="get-pricing-recommendation-flow">
  <lambda:invoke config-ref="Lambda_Config" doc:name="Get pricing
recommendation"
    functionName="recommend-pricing"
    invocationType="RequestResponse" target="pricingResult">
    <lambda:payload>#[%dw 2.0
output application/json
---
{
  product: { sku: payload.sku, category: payload.category },
  purchaseHistory: payload.recentOrders map { date: $.orderDate,
qty: $.quantity, price: $.unitPrice },
  marketConditions: { demandIndex: payload.demandIndex,
competitorAvgPrice: payload.competitorAvgPrice }
}]</lambda:payload>
  </lambda:invoke>
  <ee:transform doc:name="Flatten tiered response">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
var rec = read(vars.pricingResult, "application/json")
---
{
  recommendedPrice: rec.tiers[0].price,
  alternativeTiers: rec.tiers[1 to -1] map { label: $.label, price:
$.price }
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

```
purchaseHistory: payload.recentOrders map { ... }
```

Mapping the raw order history into exactly the field names and shape the Lambda expects — rather than

forwarding the original order objects unchanged — keeps the payload's structure an explicit, intentional contract rather than an accidental byproduct of whatever an upstream system happens to send.

```
rec.tiers[1 to -1]
```

DataWeave's range-slicing syntax pulls every tier after the first one in a single expression, cleanly separating "the recommended price" (index 0) from "the alternatives" (everything else) without a manual loop or filter.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Transform Message** to build the nested pricing-context payload.
- 2 Drag the AWS Lambda **Invoke** operation, set **Function Name** to **recommend-pricing**, and bind **Payload**.
- 3 Drag a second **Transform Message** to flatten the response using the range-slice expression.
- 4 Save and run, then verify against a Lambda test event with known input data.

TESTING IN POSTMAN

POST

https://localhost:8081/pricing/recommend

Header

Value

Content-Type

application/json

SAMPLE RESPONSE

JSON

```
{
  "recommendedPrice": 44.99,
  "alternativeTiers": [{ "label": "premium", "price": 54.99 }, {
    "label": "clearance", "price": 34.99 }]
```

BRIEFING ALL THE STEPS IN WORDS

A transform assembles a deliberately nested payload — product, purchase history, market conditions — exactly matching the shape the pricing Lambda expects, the Lambda Connector's Invoke operation calls it synchronously, and a second transform navigates the returned tiered structure back into a flat, directly usable recommendation — proving that neither the request nor the response has to be flat for Mule and Lambda to communicate cleanly.

#44

Handling Lambda Invocation Errors and Timeouts in Mule

AWS

PROBLEM STATEMENT

The tax-calculation Lambda from Problem #41 occasionally times out or throws an unhandled exception, and right now that crashes the entire checkout flow instead of falling back to a safe default.

DETAILED EXPLANATION

A Lambda function is external code the integration doesn't control, and it can fail in ways specific to Lambda itself — a cold start exceeding the invocation timeout, an unhandled runtime exception, a throttling error when concurrent executions hit the account's limit. Treating a Lambda call like any other potentially-failing external dependency, with a dedicated error branch and a sensible fallback, keeps one flaky function from taking down checkout entirely.

CONNECTORS & COMPONENTS USED

AWS

AWS Lambda Connector – Invoke

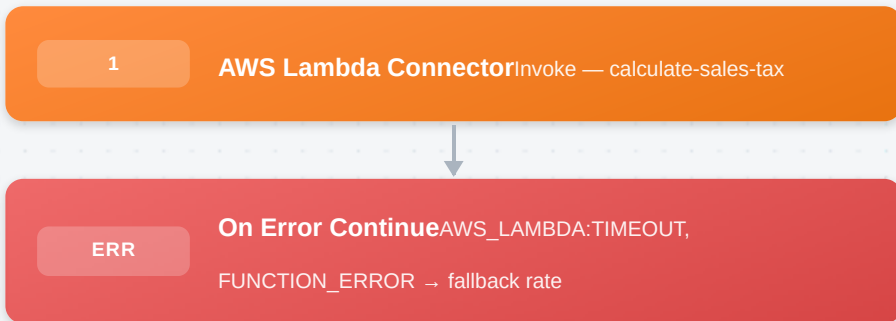
ERR

On Error Continue

STEPS TO CREATE THE MULE FLOW

- 1 Wrap the Lambda invocation in a Try scope.
- 2 Catch Lambda-specific error types (timeout, function error, throttling) separately.
- 3 Fall back to a conservative default tax rate and flag the order for manual review.

RESILIENT-TAX-CALCULATION-FLOW



MULE CONFIGURATION XML

```
<flow name="resilient-tax-calculation-flow">
  <try>
    <lambda:invoke config-ref="Lambda_Config" doc:name="Invoke tax
calculator"
      functionName="calculate-sales-tax"
      invocationType="RequestResponse"
      timeout="5000" target="taxResult">
      <lambda:payload>#[payload]</lambda:payload>
    </lambda:invoke>
    <error-handler>
      <on-error-continue type="AWS_LAMBDA:TIMEOUT,
AWS_LAMBDA:FUNCTION_ERROR, AWS_LAMBDA:THROTTLING">
        <logger level="WARN" message="#[ 'Tax Lambda failed ( ' ++
error.errorType.identifier ++ '), using fallback rate' ]"/>
        <set-variable variableName="taxResult" value="#[write({
taxAmount: payload.subtotal * 0.0825, source: 'FALLBACK_RATE' },
'application/json')]" />
      </on-error-continue>
    </error-handler>
  </try>
</flow>
```

EXPLANATION OF THE CODE

timeout="5000" on the Invoke operation

Setting an explicit client-side timeout, shorter than what the checkout flow's own caller would tolerate, guarantees the fallback path triggers on Mule's own schedule rather than waiting on Lambda's much longer default timeout budget.

source: 'FALLBACK_RATE' included in the substitute result

Tagging the fallback value as a fallback, not just silently returning a plausible-looking tax amount, is what lets a later reconciliation process identify and correct any orders that shipped with an approximate rather than exact tax calculation.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Wrap the Lambda **Invoke** operation in a **Try** scope and set its **Timeout** to **5000** ms.
- 2 Add an **On Error Continue** handler matching **AWS_LAMBDA:TIMEOUT** , **AWS_LAMBDA:FUNCTION_ERROR** , and **AWS_LAMBDA:THROTTLING** .
- 3 Inside the handler, drag a **Logger** and a **Set Variable** that computes the fallback tax amount.
- 4 Save and run, then test by pointing at a Lambda configured to always throw, and confirm the fallback path.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/calculate-tax

SAMPLE RESPONSE (LAMBDA UNAVAILABLE)

JSON

```
{ "taxAmount": 16.42, "source": "FALLBACK_RATE" }
```

BRIEFING ALL THE STEPS IN WORDS

A Try scope wraps the tax-calculation Lambda invocation with an explicit five-second timeout, and an On Error Continue handler catches Lambda-specific timeout, function, and throttling errors, substituting a clearly-labeled fallback tax rate instead of letting checkout fail outright — treating a Lambda call with the same defensive care as any other external dependency the flow doesn't control.

#45

Inserting a Record into a DynamoDB Table from a Mule Flow

AWS

PROBLEM STATEMENT

Every completed checkout needs a record in a DynamoDB order-summary table — a fast, single-digit-millisecond-latency store that a customer-facing order-status page reads from directly, separate from the system of record.

DETAILED EXPLANATION

DynamoDB is a natural fit for a read-heavy, latency-sensitive lookup like "show me this one order's status" — it's a managed key-value and document store built for that access pattern specifically, rather than the more general relational querying a system of record typically supports. Writing a purpose-built summary record here, separate from wherever the order's full transactional data lives, is what lets the customer-facing status page stay fast without querying a heavier backend system on every page load.

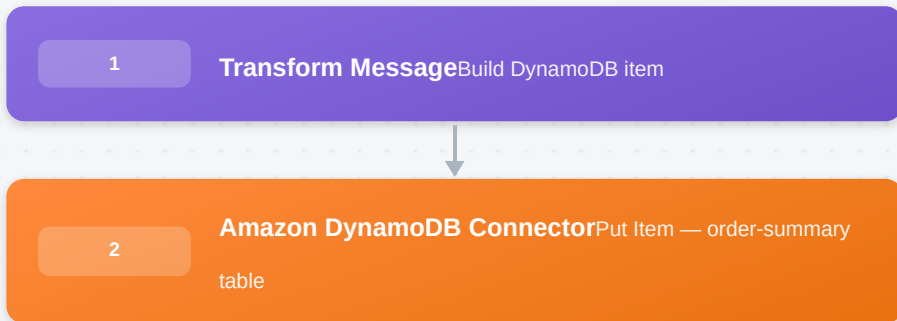
CONNECTORS & COMPONENTS USED

AWS Amazon DynamoDB Connector – Put Item

STEPS TO CREATE THE MULE FLOW

- 1 After checkout completes, build a compact order-summary item.
- 2 Call Put Item against the order-summary table.
- 3 Continue the flow — this is a fire-and-continue write, not blocking on any downstream response.

WRITE-ORDER-SUMMARY-FLOW



MULE CONFIGURATION XML

```
<flow name="write-order-summary-flow">
  <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Write
order summary"
  tableName="order-summary">
    <dynamodb:item>#[%dw 2.0
output application/json
---
{
  orderId: { S: vars.orderId },
  customerId: { S: payload.customerId },
  status: { S: 'CONFIRMED' },
  total: { N: payload.total as String },
  createdAt: { S: now() as String }
}]</dynamodb:item>
  </dynamodb:put-item>
</flow>
```

EXPLANATION OF THE CODE

`{ S: ... }` and `{ N: ... }` wrapping each attribute value

DynamoDB's native API represents every attribute as a typed value — S for string, N for number, and so

on — rather than plain JSON types, so each field in the item has to be explicitly tagged with its DynamoDB type.

```
total: { N: payload.total as String }
```

Even a numeric DynamoDB attribute is written as a string inside the N wrapper — DynamoDB parses it back into a number on its side, but the connector's item format always expects the value itself as text.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Search the Mule Palette for `Amazon DynamoDB` and drag `Put Item` onto the canvas.
- 2 Set `Table Name` to `order-summary` and write the `Item` DataWeave script with typed attribute wrappers.
- 3 Click the red flag on the DynamoDB config and enter the AWS Access Key and Secret Key.
- 4 Save and run, then confirm the item in the DynamoDB console's item explorer.

TESTING IN POSTMAN

POST

`https://localhost:8081/orders/{orderId}/confirm`

SAMPLE RESPONSE



JSON

```
{ "orderId": "8f2a1c44-...", "summaryWritten": true }
```

BRIEFING ALL THE STEPS IN WORDS

A transform builds a compact order-summary item with each field explicitly typed for DynamoDB, and the DynamoDB Connector's Put Item operation writes it into the order-summary table — giving a customer-facing status page a fast, purpose-built record to read from instead of querying the full system of record on every page view.

PROBLEM STATEMENT

The order-status page needs every order for a given customer, sorted by date, without scanning the entire order-summary table — which would get slower and more expensive as the table grows.

DETAILED EXPLANATION

DynamoDB's Query operation is efficient precisely because it's restricted: it requires an exact match on the table's partition key, and optionally a range condition on the sort key, which lets DynamoDB jump directly to the relevant partition instead of examining every item in the table the way a Scan (Problem #50) does.

Designing the table with `customerId` as the partition key and `createdAt` as the sort key is what makes "this customer's orders, newest first" a single efficient Query rather than a table-wide filter.

CONNECTORS & COMPONENTS USED

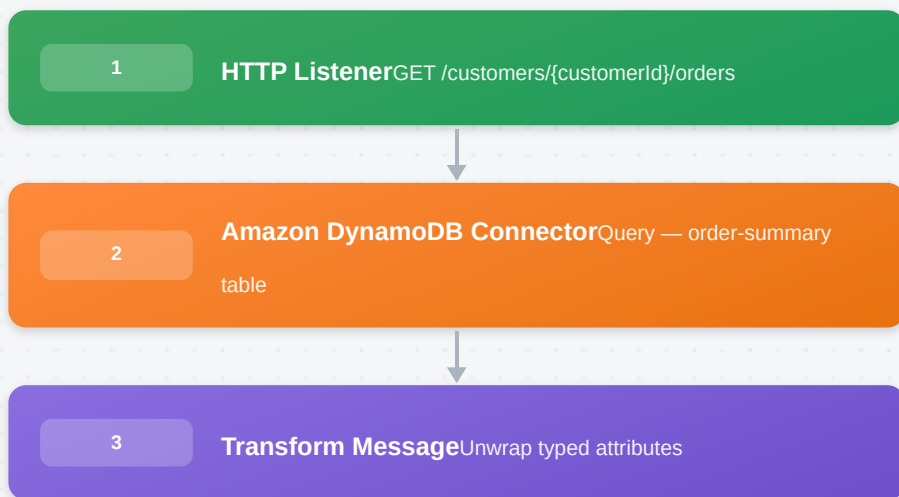
CORE HTTP Listener

AWS Amazon DynamoDB Connector – Query

STEPS TO CREATE THE MULE FLOW

- 1 Accept a customer ID over HTTP.
- 2 Query the order-summary table's partition key for that exact customer ID.
- 3 Return the matching orders sorted newest-first.

GET-CUSTOMER-ORDERS-FLOW



MULE CONFIGURATION XML

```

<flow name="get-customer-orders-flow">
  <http:listener path="/customers/{customerId}/orders" config-
ref="HTTP_Listener_config"/>
  <dynamodb:query config-ref="DynamoDB_Config" doc:name="Query
orders by customer"
    tableName="order-summary"
    keyConditionExpression="customerId = :cid"
    scanIndexForward="false" target="queryResult">
    <dynamodb:expression-attribute-values>#[{ ':cid': { S:
attributes.uriParams.customerId } }]</dynamodb:expression-attribute-
values>
  </dynamodb:query>
  <ee:transform doc:name="Unwrap typed items">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
vars.queryResult.items map {
  orderId: $.orderId.S,
  status: $.status.S,
  total: $.total.N as Number,
  createdAt: $.createdAt.S
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

scanIndexForward="false"

DynamoDB returns Query results in ascending sort-key order by default; setting this to false reverses that, so with **createdAt** as the sort key, the newest orders come back first without any additional client-side sorting.

\$.total.N as Number in the unwrap transform

Converting the DynamoDB-native numeric string back into an actual DataWeave

number is what lets the returned JSON be used directly by any client — without this cast, every consumer of this endpoint would need to know to parse `total` from a string itself.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an `HTTP Listener` onto the canvas and set `Path` to `/customers/{customerId}/orders`.
- 2 Drag the DynamoDB `Query` operation, set `Table Name`, `Key Condition Expression`, and `Scan Index Forward` to `false`.
- 3 Bind `Expression Attribute Values` to the customer ID from the URI parameter.
- 4 Drag a `Transform Message` to unwrap the typed attributes, then save and run.

TESTING IN POSTMAN

GET

`https://localhost:8081/customers/CUST-5521/orders`

SAMPLE RESPONSE

JSON

```
[
  { "orderId": "8f2a1c44-...", "status": "CONFIRMED", "total": 216.41,
    "createdAt": "2026-08-28T09:12:03Z" }
]
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a customer ID, the DynamoDB Connector's Query operation matches it against the table's partition key and returns results newest-first via the sort key, and a transform unwraps DynamoDB's typed attribute format into plain JSON — an efficient, partition-scoped lookup that stays fast regardless of how large the overall table grows.

PROBLEM STATEMENT

An order's status should only transition from PENDING to SHIPPED, never the reverse, and never from a CANCELLED state — but a plain update would happily overwrite the status regardless of what it currently is.

DETAILED EXPLANATION

An unconditional update is a race condition waiting to happen: if two processes update the same item around the same time, whichever writes last simply wins, even if that means moving a cancelled order back to shipped. DynamoDB's conditional updates solve this directly at the database level — a condition expression attached to the update means the write only applies if the item currently matches that condition, and fails cleanly (rather than silently succeeding incorrectly) otherwise.

CONNECTORS & COMPONENTS USED

AWS

Amazon DynamoDB Connector – Update Item

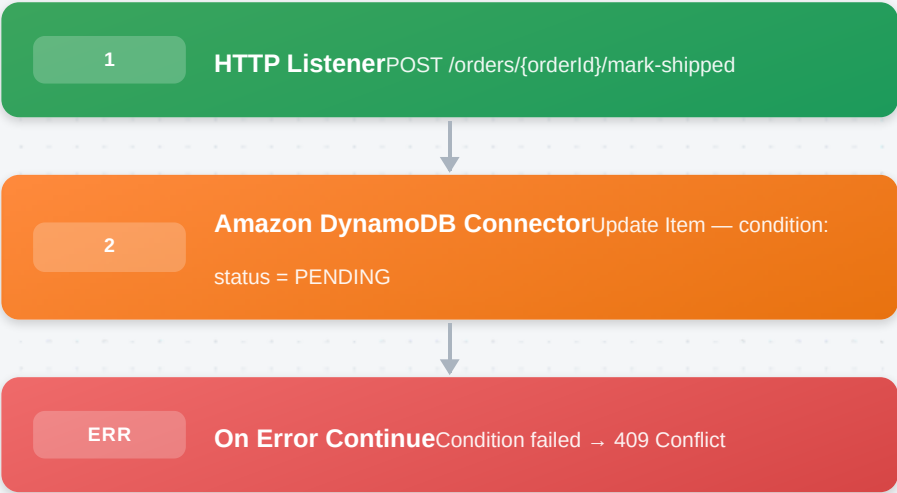
ERR

On Error Continue

STEPS TO CREATE THE MULE FLOW

- 1 Accept a request to mark an order shipped.
- 2 Call Update Item with a condition expression requiring the current status to be PENDING.
- 3 Catch the conditional-check-failed error and return a clear, specific rejection.

SHIP-ORDER-CONDITIONALLY-FLOW



MULE CONFIGURATION XML


```

<flow name="ship-order-conditionally-flow">
  <http:listener path="/orders/{orderId}/mark-shipped" config-
ref="HTTP_Listener_config"/>
  <try>
    <dynamodb:update-item config-ref="DynamoDB_Config"
doc:name="Transition to SHIPPED"
    tableName="order-summary"
    updateExpression="SET #s = :newStatus"
    conditionExpression="#s = :expectedStatus">
      <dynamodb:key>#[{ orderId: { S: attributes.uriParams.orderId }
}]/></dynamodb:key>
      <dynamodb:expression-attribute-names>#[{ '#s': 'status' }]/>
</dynamodb:expression-attribute-names>
      <dynamodb:expression-attribute-values>#[{ ':newStatus': { S:
'SHIPPED' }, ':expectedStatus': { S: 'PENDING' } }]/>
</dynamodb:expression-attribute-values>
    </dynamodb:update-item>
    <error-handler>
      <on-error-continue
type="AWS_DYNAMODB:CONDITIONAL_CHECK_FAILED">
        <set-variable variableName="httpStatus" value="409"/>
        <set-payload value="#[{ error: 'ORDER_NOT_IN_PENDING_STATE'
}]" />
      </on-error-continue>
    </error-handler>
  </try>
</flow>

```

EXPLANATION OF THE CODE

#s used as an alias for status via expression-attribute-names

status is a reserved word in DynamoDB's expression syntax, so it can't be referenced directly

in an expression; the `#s` alias is DynamoDB's required workaround for using a reserved attribute name in any expression.

```
conditionExpression="#s = :expectedStatus"
```

This is what actually enforces the state transition rule — the update only applies at all if the item's current status genuinely equals PENDING at the moment DynamoDB evaluates the write, atomically, with no window for a race condition to sneak through.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an `HTTP Listener` onto the canvas and set `Path` to `/orders/{orderId}/mark-shipped`.
- 2 Wrap the DynamoDB `Update Item` operation in a `Try` scope, and set `Update Expression` and `Condition Expression`.
- 3 Set `Expression Attribute Names` and `Expression Attribute Values` as shown.
- 4 Add an `On Error Continue` handler for `AWS_DYNAMODB:CONDITIONAL_CHECK_FAILED` returning a 409, then save and run.

TESTING IN POSTMAN

POST

`https://localhost:8081/orders/8f2a1c44-.../mark-shipped`

SAMPLE RESPONSE (ORDER ALREADY SHIPPED)

JSON

```
{ "error": "ORDER_NOT_IN_PENDING_STATE" }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a request to mark an order shipped, the DynamoDB Connector's Update Item operation only applies the transition if the item's status is currently PENDING — enforced atomically by DynamoDB's own condition expression — and an On Error Continue handler catches a failed condition check and returns a clear 409 instead of a raw database error, closing off a race condition an unconditional update would have left wide open.

#48 Deleting an Item from DynamoDB After Processing

AWS

PROBLEM STATEMENT

A DynamoDB table used to track in-flight idempotency keys for the payment flow needs each key removed once its associated payment has been fully settled, so the table doesn't grow unbounded with keys nobody needs anymore.

DETAILED EXPLANATION

An idempotency-key table exists specifically to detect and reject duplicate requests while the original is still being processed; once processing is genuinely finished, the key has done its job and keeping it around indefinitely only adds unnecessary storage and read cost. Deleting it explicitly — rather than relying purely on a DynamoDB TTL attribute, which expires items only approximately, within a window of up to 48 hours — keeps the table's active size tightly bounded to genuinely in-flight requests.

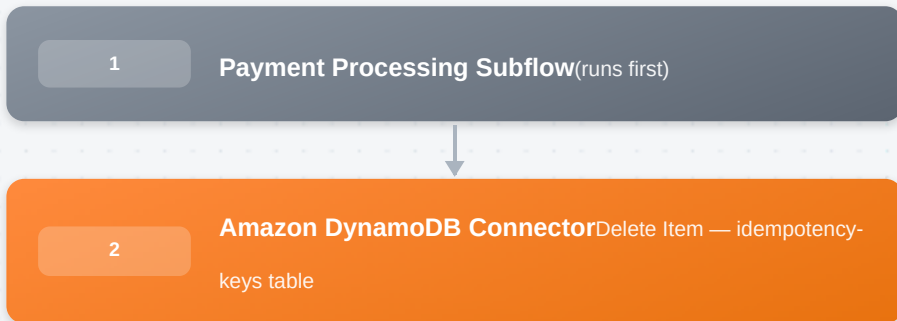
CONNECTORS & COMPONENTS USED

AWS Amazon DynamoDB Connector – Delete Item

STEPS TO CREATE THE MULE FLOW

- 1 Complete payment processing for a given idempotency key.
- 2 Call Delete Item against the idempotency-keys table for that key.
- 3 Confirm removal before returning the payment result.

CLEANUP-IDEMPOTENCY-KEY-FLOW



MULE CONFIGURATION XML

```
<flow name="cleanup-idempotency-key-flow">
  <flow-ref name="process-payment-subflow"/>
  <dynamodb:delete-item config-ref="DynamoDB_Config"
doc:name="Remove idempotency key"
  tableName="idempotency-keys">
    <dynamodb:key>#{ idempotencyKey: { S: vars.idempotencyKey } } ]
  </dynamodb:key>
  </dynamodb:delete-item>
</flow>
```

EXPLANATION OF THE CODE

`flow-ref to the payment subflow placed before the delete`

Deleting the idempotency key only after payment processing has actually completed — not before — guarantees a concurrent duplicate request within the same processing window still finds the key present and gets correctly rejected as a duplicate.

dynamodb:key using only the table's partition key

Delete Item only ever needs the item's key attributes, not its full set of fields, which keeps this cleanup call minimal — there's no need to know or fetch anything else about the record being removed.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Flow Reference** to the payment-processing subflow.
- 2 Drag the DynamoDB **Delete Item** operation, set **Table Name** to **idempotency-keys**, and bind **Key** to the current idempotency key.
- 3 Save and run, then confirm in the DynamoDB console that the key disappears once payment completes.

TESTING IN POSTMAN

POST

https://localhost:8081/payments/process

Header

Value

Idempotency-Key

a1b2c3d4-...

SAMPLE RESPONSE



JSON

```
{ "paymentStatus": "SETTLED", "idempotencyKeyRemoved": true }
```

BRIEFING ALL THE STEPS IN WORDS

A Flow Reference runs the actual payment processing first, and only once it completes does the DynamoDB Connector's Delete Item operation remove that request's idempotency key — keeping the table's active footprint limited to genuinely in-flight requests instead of relying solely on DynamoDB's looser, multi-hour TTL expiration.

PROBLEM STATEMENT

A bulk order-import job needs to write summary records for 500 orders into DynamoDB, and writing them one Put Item call at a time is far slower than necessary.

DETAILED EXPLANATION

DynamoDB's Batch Write Item operation supports up to 25 put or delete requests in a single call, similar in spirit to SQS's batch send from Problem #30, and offers the same kind of throughput improvement. The complication worth knowing about is that DynamoDB can return unprocessed items when it throttles part of a batch under heavy write load — a caller that doesn't check for and retry those has silently lost writes, even though the overall API call reported success.

CONNECTORS & COMPONENTS USED

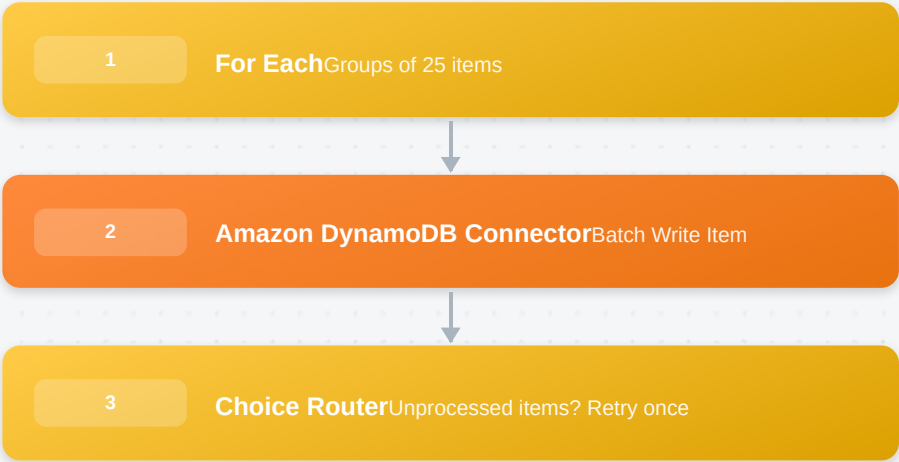
CORE For Each (batches of 25)

AWS Amazon DynamoDB Connector – Batch Write Item

STEPS TO CREATE THE MULE FLOW

- 1 Split the 500 order summaries into groups of 25.
- 2 Call Batch Write Item for each group.
- 3 Retry any unprocessed items DynamoDB returns.

BULK-WRITE-ORDER-SUMMARIES-FLOW



MULE CONFIGURATION XML

```

<flow name="bulk-write-order-summaries-flow">
  <foreach collection="#[payload divideBy 25]">
    <dynamodb:batch-write-item config-ref="DynamoDB_Config"
doc:name="Batch write 25 summaries" target="batchResult">
      <dynamodb:request-items>#[{
        'order-summary': payload map {
          PutRequest: { Item: { orderId: { S: $.orderId }, customerId: {
S: $.customerId }, total: { N: $.total as String } } }
        }
      ]</dynamodb:request-items>
    </dynamodb:batch-write-item>
    <choice>
      <when expression="#[not
isEmpty(vars.batchResult.unprocessedItems)]">
        <dynamodb:batch-write-item config-ref="DynamoDB_Config"
doc:name="Retry unprocessed items once">
          <dynamodb:request-items>#
[vars.batchResult.unprocessedItems]</dynamodb:request-items>
        </dynamodb:batch-write-item>
      </when>
    </choice>
  </foreach>
</flow>

```

EXPLANATION OF THE CODE

```
'order-summary': payload map { PutRequest: ... }
```

DynamoDB's batch write request format is keyed by table name, allowing a single batch call to write across multiple tables at once; here every entry targets the same order-summary table, wrapped individually as a **PutRequest**.

```
not isEmpty(vars.batchResult.unprocessedItems)
```

Checking for unprocessed items and retrying them once is what prevents a

throttled batch write from silently dropping records — without this check, the flow would report success on a call that actually only wrote part of the batch.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **For Each** scope with **Collection** set to `#[payload divideBy 25]`.
- 2 Inside it, drag the DynamoDB **Batch Write Item** operation and write the **Request Items** mapping, setting **Target Variable** to `batchResult`.
- 3 Drag a **Choice** router checking for `unprocessedItems`, with a second **Batch Write Item** call inside the **when** branch.
- 4 Save and run, then test against a bulk import large enough to require multiple batches.

TESTING IN POSTMAN

POST

`https://localhost:8081/orders/bulk-import-summaries`

SAMPLE RESPONSE

JSON

```
{ "totalItems": 500, "batchesSent": 20 }
```

BRIEFING ALL THE STEPS IN WORDS

A For Each loop splits the 500 order summaries into groups of 25 using DataWeave's `divideBy` function, the DynamoDB Connector's Batch Write Item operation writes each group in a single call, and a Choice router retries any items DynamoDB reports as unprocessed — cutting the write time down substantially while still catching the throttling edge case that batch writes are known to produce under load.

PROBLEM STATEMENT

Closing out Section 1: an internal operations tool needs to find every order still in PENDING status older than 48 hours — a query pattern that doesn't fit the partition-key-based Query operation, since it's not scoped to any one customer.

DETAILED EXPLANATION

Scan reads every item in the table and applies a filter expression afterward, which is fundamentally more expensive than Query's targeted partition lookup — it consumes read capacity for the entire table, not just the matching items, and its cost grows linearly with table size. It's still the right tool here, because "every stale pending order, across all customers" genuinely isn't a partition-scoped question; the discipline that matters is treating Scan as an occasional operational tool, not something called on a hot path serving live customer traffic.

CONNECTORS & COMPONENTS USED

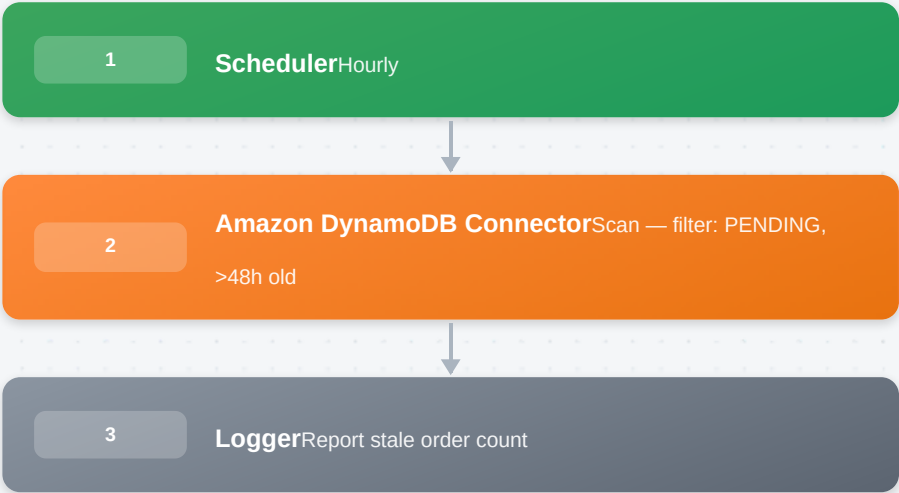
CORE Scheduler

AWS Amazon DynamoDB Connector – Scan

STEPS TO CREATE THE MULE FLOW

- 1 Run hourly, not on any customer-facing path.
- 2 Scan the order-summary table with a filter for status PENDING and createdAt older than 48 hours.
- 3 Report the stale orders for operations to review.

FIND-STALE-PENDING-ORDERS-FLOW



MULE CONFIGURATION XML

```

<flow name="find-stale-pending-orders-flow">
  <scheduler doc:name="Hourly">
    <scheduling-strategy>
      <fixed-frequency frequency="1" timeUnit="HOURS"/>
    </scheduling-strategy>
  </scheduler>
  <dynamodb:scan config-ref="DynamoDB_Config" doc:name="Scan for
stale pending orders"
    tableName="order-summary"
    filterExpression="#s = :pending AND createdAt < :cutoff"
    target="scanResult">
    <dynamodb:expression-attribute-names>#{[ '#s': 'status' ]]}
  </dynamodb:expression-attribute-names>
    <dynamodb:expression-attribute-values>#{[ {
    ':pending': { S: 'PENDING' },
    ':cutoff': { S: (now() - |PT48H|) as String }
  }]}</dynamodb:expression-attribute-values>
  </dynamodb:scan>
  <logger level="WARN" message="#[(sizeOf(vars.scanResult.items) as
String) ++ ' stale pending orders found']"/>
</flow>

```

EXPLANATION OF THE CODE

filterExpression applied after DynamoDB reads every item

Unlike Query's key condition, which limits what DynamoDB reads in the first place, Scan's filter expression is applied only after every item in the table has already been read and paid for in consumed read capacity — it narrows the returned results, not the actual work done.

```
now() - |PT48H| computing the 48-hour cutoff
```

Subtracting an ISO-8601 period directly from the current datetime gives a clean, readable cutoff calculation, avoiding manual millisecond arithmetic to express "48 hours ago."

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Scheduler** onto the canvas and set it to run **Every 1 Hour**.
- 2 Drag the DynamoDB **Scan** operation, set **Table Name**, and write the **Filter Expression** and attribute maps.
- 3 Drag a **Logger** to report the count found, then save and run.
- 4 Note in the flow's documentation that this operation should never be called from a customer-facing endpoint, only on this scheduled path.

TESTING IN POSTMAN

This flow is Scheduler-triggered. Expose an admin endpoint to run it on demand for testing.

POST

`https://localhost:8081/admin/stale-orders/run-now`

CONSOLE OUTPUT



LOG

WARN 3 stale pending orders found

BRIEFING ALL THE STEPS IN WORDS

An hourly Scheduler runs the DynamoDB Connector's Scan operation across the entire order-summary table with a filter for pending orders older than 48 hours, and a Logger reports how many were found — closing Section 1 by using Scan deliberately and sparingly, for the one genuinely table-wide operational question a partition-scoped Query could never efficiently answer.

SECTION 2 Kinesis Streaming & Step Functions Orchestration (Problems 51–60)

#51 Reading Records from a Kinesis Data Stream in Real Time

AWS

PROBLEM STATEMENT

A clickstream-analytics platform publishes user browsing events to a Kinesis Data Stream continuously, and a Mule flow needs to consume them in near-real-time to update a live "trending products" feed.

DETAILED EXPLANATION

Kinesis Data Streams is built for exactly this kind of high-throughput, ordered, real-time record ingestion — a stream is divided into shards, each an independently-readable, ordered sequence of records, and a consumer reads by repeatedly requesting the next batch starting from a shard iterator. Unlike SQS, records aren't removed once read; multiple independent consumers can each read the same stream at their own pace, which is exactly the model a "trending products" feed needs when it's just one of several systems consuming the same clickstream.

CONNECTORS & COMPONENTS USED

AWS

Amazon Kinesis Connector – On New Record

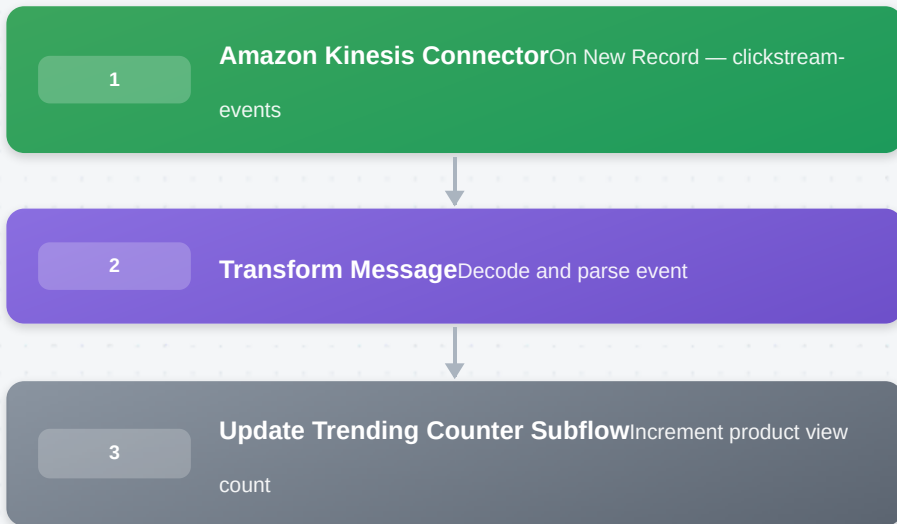
DW

Transform Message

STEPS TO CREATE THE MULE FLOW

- 1 Configure the Kinesis connector's On New Record source against the clickstream-events stream.
- 2 Parse each record's base64-encoded data back into a structured event.
- 3 Update the trending-products counter for the viewed product.

CONSUME-CLICKSTREAM-FLOW



MULE CONFIGURATION XML

```
<flow name="consume-clickstream-flow">
  <kinesis:listener config-ref="Kinesis_Config" doc:name="Watch
clickstream stream"
    streamName="clickstream-events" shardIteratorType="LATEST"/>
  <ee:transform doc:name="Decode record">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
read(payload.data as String {encoding: "base64"},
"application/json")]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <flow-ref name="update-trending-counter-subflow"/>
</flow>
```

EXPLANATION OF THE CODE

```
shardIteratorType="LATEST"
```

Starting from LATEST means this consumer only sees records published after it started listening — the right choice for a live trending feed, which cares about current activity, not replaying the stream's entire history from the beginning.

```
payload.data as String {encoding: "base64"}
```

Kinesis delivers each record's data as base64-encoded bytes regardless of what was originally published, so decoding it back to a string — before parsing it as JSON — is a required first step for every Kinesis consumer, not specific to this event type.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Search the Mule Palette for `Amazon Kinesis` and drag `On New Record` as the flow's source.
- 2 Set `Stream Name` to `clickstream-events` and `Shard Iterator Type` to `LATEST`.
- 3 Drag a `Transform Message` to base64-decode and parse the record data.
- 4 Drag a `Flow Reference` to the trending-counter update logic, then save and run.

TESTING IN POSTMAN

Stream-triggered. Publish a test record via the AWS CLI or Problem #52's flow, then watch this flow's console output.

POST

```
(AWS CLI) aws kinesis put-record --stream-name clickstream-events ...
```

CONSOLE OUTPUT



LOG

INFO Trending counter incremented for product WIDGET-01

BRIEFING ALL THE STEPS IN WORDS

The Kinesis Connector's On New Record source reads new browsing events from the clickstream stream in real time, a transform decodes each record's base64 payload back into structured JSON, and a subflow increments the trending-products counter — one of potentially several independent consumers reading the same continuously-flowing stream at its own pace.

#52

Publishing Records to a Kinesis Data Stream from Mule

AWS

PROBLEM STATEMENT

Every product page view on the customer-facing site needs to be published to the clickstream-events stream, from the same Mule API layer that serves the page data itself.

DETAILED EXPLANATION

Publishing to Kinesis is the producer side of Problem #51's consumer, and the one detail that genuinely matters is the partition key: Kinesis uses it to decide which shard a record lands on, and records sharing a partition key are guaranteed to stay in order relative to each other on that shard. Choosing a partition key that distributes reasonably evenly across shards — here, a hash of the session ID — avoids concentrating all the write throughput onto one "hot" shard.

CONNECTORS & COMPONENTS USED

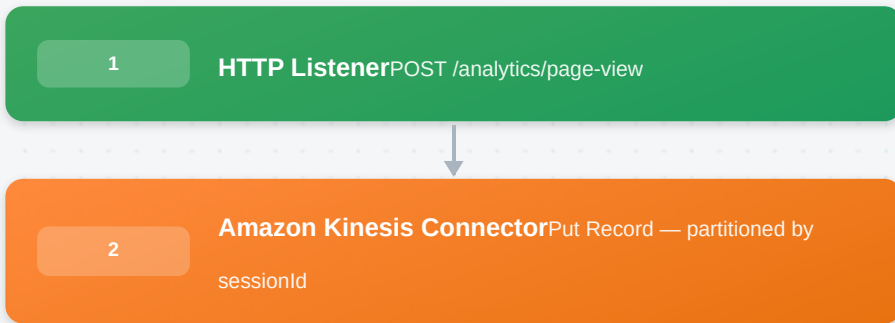
CORE HTTP Listener

AWS Amazon Kinesis Connector – Put Record

STEPS TO CREATE THE MULE FLOW

- 1 Accept a page-view event over HTTP.
- 2 Publish it to the clickstream-events stream, keyed by session ID.
- 3 Return without waiting on any downstream consumer.

PUBLISH-PAGE-VIEW-FLOW



MULE CONFIGURATION XML

```
<flow name="publish-page-view-flow">
  <http:listener path="/analytics/page-view" config-
ref="HTTP_Listener_config"/>
  <kinesis:put-record config-ref="Kinesis_Config" doc:name="Publish
page view"
  streamName="clickstream-events"
  partitionKey="#[payload.sessionId]">
    <kinesis:data>#[write(payload, "application/json") as String
{encoding: "base64"}]</kinesis:data>
  </kinesis:put-record>
</flow>
```

EXPLANATION OF THE CODE

`partitionKey="#[payload.sessionId]"`

Using session ID as the partition key means every page-view event from one browsing session stays strictly ordered relative to the others, which matters if a downstream consumer ever needs to reconstruct a session's click path in the order it actually happened.

`write(payload, "application/json") as String {encoding: "base64"}`

The producer side has to perform the encoding step

that Problem #51's consumer later reverses — Kinesis's Data field is always base64, so the JSON event has to be explicitly encoded before being sent, not left as plain text.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/analytics/page-view`.
- 2 Drag the Kinesis **Put Record** operation, set **Stream Name** and **Partition Key**.
- 3 Write the **Data** field's DataWeave expression to JSON-encode and base64-encode the payload.
- 4 Save and run, then confirm the record appears via Problem #51's consumer flow.

TESTING IN POSTMAN

POST

`https://localhost:8081/analytics/page-view`

Header

Value

Content-Type

application/json

SAMPLE REQUEST BODY

JSON

```
{ "sessionId": "sess-77123", "productSku": "WIDGET-01", "viewedAt":  
  "2026-08-28T09:30:00Z" }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a page-view event, and the Kinesis Connector's Put Record operation publishes it to the clickstream-events stream base64-encoded and partitioned by session ID — keeping one session's events in order on the same shard while spreading overall write traffic evenly across the stream.

#53

Handling Kinesis Shard-Iterator Expiration During Continuous Polling

AWS

PROBLEM STATEMENT

A custom Kinesis consumer built around manual Get Shard Iterator and Get Records calls occasionally stops receiving new records after several minutes of low activity, requiring a full restart to recover.

DETAILED EXPLANATION

A shard iterator obtained from Get Shard Iterator is only valid for five minutes; if Get Records isn't called again before it expires — which can happen during a slow period, or if error handling pauses the consumer — the next call fails with an expired-iterator error, and continuing to read requires fetching a completely new iterator. The Mule Kinesis connector's built-in On New Record source handles this automatically, refreshing the shard iterator internally, which is exactly why it's the recommended approach over manually managing Get Shard Iterator and Get Records calls.

CONNECTORS & COMPONENTS USED

AWS

Amazon Kinesis Connector – On New Record

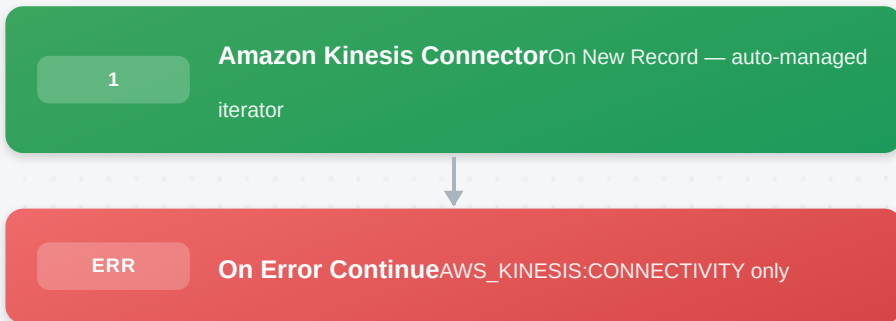
ERR

On Error Continue

STEPS TO CREATE THE MULE FLOW

- 1 Replace manual Get Shard Iterator / Get Records calls with the On New Record source.
- 2 Let the connector manage iterator refresh internally.
- 3 Add error handling only for genuine stream-level errors, not iterator management.

RESILIENT-KINESIS-CONSUMER-FLOW



MULE CONFIGURATION XML

```
<flow name="resilient-kinesis-consumer-flow">
  <kinesis:listener config-ref="Kinesis_Config" doc:name="Watch
stream (auto iterator management)"
    streamName="clickstream-events" shardIteratorType="LATEST"
    pollingFrequencyMillis="1000">
    <reconnect frequency="5000" count="10"/>
  </kinesis:listener>
  <flow-ref name="process-clickstream-record-subflow"/>
  <error-handler>
    <on-error-continue type="AWS_KINESIS:CONNECTIVITY">
      <logger level="WARN" message="Transient Kinesis connectivity
issue; listener will reconnect automatically"/>
    </on-error-continue>
  </error-handler>
</flow>
```

EXPLANATION OF THE CODE

`reconnect frequency="5000" count="10"`

Configuring the listener's reconnection strategy directly means a transient connectivity blip — not an expired iterator, since the connector already manages that — is retried automatically up to ten

times, five seconds apart, before the source gives up.

no manual shard iterator logic anywhere in this flow

The entire fix for the original problem is structural: by using `kinesis:listener` instead of hand-rolled `get-shard-iterator` and `get-records` calls, iterator expiration simply becomes something the connector handles internally rather than something the flow's own code needs to detect and recover from.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Replace any manual `Get Shard Iterator` / `Get Records` operations with the Kinesis `On New Record` source.
- 2 Set `Polling Frequency Millis` to `1000` and configure `Reconnection Strategy` with a frequency and count.
- 3 Add an `On Error Continue` handler scoped specifically to `AWS_KINESIS:CONNECTIVITY`.
- 4 Save and run, then leave the flow idle for over five minutes and confirm it still picks up new records without a restart.

TESTING IN POSTMAN

Stream-triggered. Publish a record, wait six minutes of idle time, then publish another and confirm both are received.

POST

```
(AWS CLI) aws kinesys put-record --stream-name clickstream-events ...
```

CONSOLE OUTPUT

LOG

```
INFO Record received after 6 minutes idle, iterator managed automatically
```

BRIEFING ALL THE STEPS IN WORDS

Replacing manual shard-iterator management with the Kinesis Connector's On New Record source lets Mule handle iterator refresh internally, and a narrowly-scoped error handler and reconnection strategy cover genuine connectivity blips — eliminating the class of bug where a quiet stream silently stops delivering records until the whole consumer is restarted.

#54

Processing Kinesis Stream Records in Batches with Checkpointing

AWS

PROBLEM STATEMENT

Processing clickstream records one at a time is too slow to keep up with peak traffic, and if the consumer crashes mid-stream, it currently has no way to resume from where it left off rather than reprocessing from the beginning.

DETAILED EXPLANATION

Batching records together before processing amortizes per-record overhead across many records at once, which matters when throughput, not per-record latency, is the priority. Checkpointing — recording the sequence number of the last successfully processed record, typically in DynamoDB — is what makes a restart resume from that point instead of either reprocessing the entire stream's history or, worse, silently skipping ahead and losing records.

CONNECTORS & COMPONENTS USED

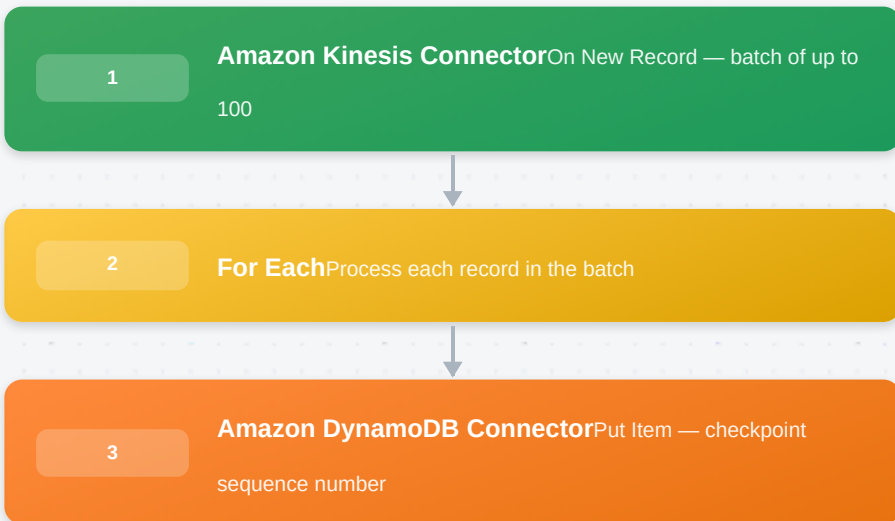
AWS Amazon Kinesis Connector – On New Record (batch)

AWS Amazon DynamoDB Connector – Put Item (checkpoint)

STEPS TO CREATE THE MULE FLOW

- 1 Configure the Kinesis listener to deliver records in batches rather than one at a time.
- 2 Process the entire batch.
- 3 Once the batch succeeds, write the last record's sequence number to a checkpoint table.

BATCH-PROCESS-WITH-CHECKPOINT-FLOW



MULE CONFIGURATION XML


```

<flow name="batch-process-with-checkpoint-flow">
  <kinesis:listener config-ref="Kinesis_Config" doc:name="Watch
stream in batches"
    streamName="clickstream-events" shardIteratorType="LATEST"
    maxRecordsPerBatch="100"/>
  <foreach collection="#[payload]">
    <flow-ref name="process-clickstream-record-subflow"/>
  </foreach>
  <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Write
checkpoint"
    tableName="kinesis-checkpoints">
    <dynamodb:item>#[%dw 2.0
output application/json
---
{
  streamName: { S: 'clickstream-events' },
  lastSequenceNumber: { S: (payload[-1]).sequenceNumber },
  checkpointedAt: { S: now() as String }
}]</dynamodb:item>
  </dynamodb:put-item>
</flow>

```

EXPLANATION OF THE CODE

`maxRecordsPerBatch="100"`

Configuring the listener to deliver up to 100 records per invocation, rather than one flow execution per record, is what actually reduces the per-record overhead — the For Each loop still processes each one, but flow-level costs like connector setup are paid once per batch instead of once per record.

`(payload[-1]).sequenceNumber as the checkpoint value`

Checkpointing only the last record's sequence number, and only after the entire batch has been successfully processed, means a restart resumes right after the last confirmed record — DataWeave's negative

indexing makes grabbing that final record from the batch a single expression.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 On the Kinesis `On New Record` source, set `Max Records Per Batch` to `100`.
- 2 Drag a `For Each` scope to process every record in the delivered batch.
- 3 After the loop, drag the DynamoDB `Put Item` operation to write the checkpoint using the batch's final sequence number.
- 4 Save and run, then confirm the checkpoint table updates after each batch.

TESTING IN POSTMAN

Stream-triggered. Publish several records in quick succession and confirm they're delivered as one batch with a single checkpoint write.

POST

```
(AWS CLI) aws kinesis put-records --stream-name clickstream-events ...
```

CONSOLE OUTPUT

LOG

```
INFO Processed batch of 47 records, checkpoint written at sequence 4955...
```

BRIEFING ALL THE STEPS IN WORDS

The Kinesis Connector's On New Record source delivers records in batches of up to 100 rather than one at a time, a For Each loop processes every record in the batch, and the DynamoDB Connector's Put Item operation checkpoints the batch's final sequence number only once everything succeeded — giving a crashed consumer a precise point to resume from instead of reprocessing the entire stream's history.

Starting an AWS Step Functions State Machine Execution from Mule

AWS

PROBLEM STATEMENT

Order fulfillment involves a multi-step process — reserve inventory, charge payment, notify warehouse, each with its own retry and compensation logic — that's cleaner to model as an AWS Step Functions state machine than as one long Mule flow.

DETAILED EXPLANATION

Step Functions is purpose-built for exactly this kind of multi-step orchestration: a visual state machine with built-in retry, catch, and parallel-branch semantics, where each state's execution history is automatically tracked and visible in the AWS Console. For a fulfillment process complex enough to need its own error-recovery logic at each step, letting Step Functions own that orchestration — while Mule simply starts the execution and later checks on it — is often clearer than encoding the same branching logic as nested Mule error handlers.

CONNECTORS & COMPONENTS USED

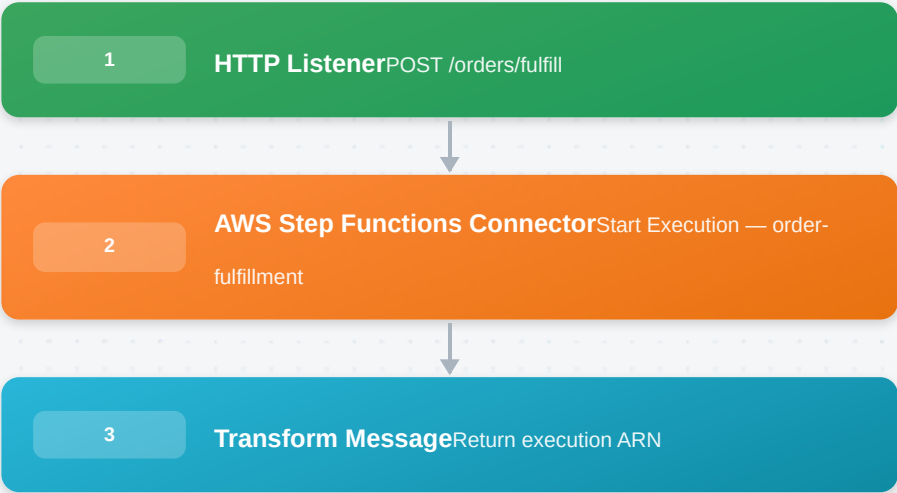
CORE HTTP Listener

AWS AWS Step Functions Connector – Start Execution

STEPS TO CREATE THE MULE FLOW

- 1 Accept a checkout request over HTTP.
- 2 Start an execution of the order-fulfillment state machine with the order as input.
- 3 Return the execution ARN so the caller can check on progress later.

START-FULFILLMENT-EXECUTION-FLOW



MULE CONFIGURATION XML

```

<flow name="start-fulfillment-execution-flow">
  <http:listener path="/orders/fulfill" config-
ref="HTTP_Listener_config"/>
  <step-functions:start-execution config-ref="StepFunctions_Config"
doc:name="Start fulfillment workflow"
  stateMachineArn="arn:aws:states:us-east-
1:123456789012:stateMachine:order-fulfillment"
  name="#['fulfillment-' ++ payload.orderId]"
target="executionResult">
    <step-functions:input>#[write(payload, "application/json")]
  </step-functions:input>
  </step-functions:start-execution>
  <ee:transform doc:name="Return execution reference">
    <ee:message>
      <ee:set-payload>#[%dw 2.0 output application/json --- {
executionArn: vars.executionResult.executionArn, status: 'STARTED'
}]]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

```
name="#['fulfillment-' ++ payload.orderId]"
```

Step Functions execution names must be unique within a state machine over a rolling window, and deriving the name from the order ID both satisfies that requirement and means a duplicate start request for the same order fails predictably rather than silently starting a second, redundant execution.

```
step-functions:input as a JSON string via write()
```

Like EventBridge's detail field in Problem #33, Step Functions expects its execution input as a JSON-encoded string rather than a nested object, since the API itself treats the input as an opaque string

it passes through to the state machine.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to **/orders/fulfill**.
- 2 Search the Mule Palette for **AWS Step Functions**, drag **Start Execution**, and set **State Machine Arn**, **Name**, and **Input**.
- 3 Set **Target Variable** to **executionResult**.
- 4 Drag a **Transform Message** to return the execution ARN, then save and run.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/fulfill

Header	Value
Content-Type	application/json

SAMPLE RESPONSE

JSON

```
{ "executionArn": "arn:aws:states:us-east-1:123456789012:execution:order-fulfillment:fulfillment-ORD-991",  
  "status": "STARTED" }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a checkout request, the Step Functions Connector's Start Execution operation kicks off the order-fulfillment state machine with a deterministic, order-derived execution name, and a transform returns the execution's ARN — handing off the multi-step orchestration entirely to Step Functions, with Mule's role limited to starting it and, later, checking on it.

PROBLEM STATEMENT

The customer-facing order-status page needs to show whether the fulfillment workflow started in Problem #55 has completed, failed, or is still running — without the original checkout flow blocking on it.

DETAILED EXPLANATION

Step Functions executions run independently once started, potentially taking anywhere from seconds to hours depending on the workflow, so checking status is naturally a separate, later concern from starting the execution. Describe Execution is a lightweight, read-only call that returns the current status and, once finished, the final output or error — exactly what a status page needs to poll periodically without any coupling to the original flow that kicked things off.

CONNECTORS & COMPONENTS USED

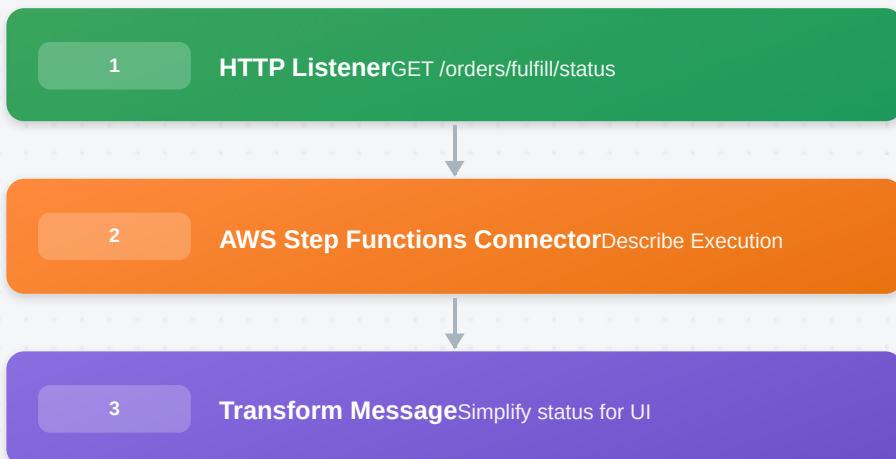
CORE HTTP Listener

AWS AWS Step Functions Connector – Describe Execution

STEPS TO CREATE THE MULE FLOW

- 1 Accept an execution ARN over HTTP.
- 2 Call Describe Execution for that ARN.
- 3 Return a simplified status the front end can render directly.

CHECK-FULFILLMENT-STATUS-FLOW



```

<flow name="check-fulfillment-status-flow">
  <http:listener path="/orders/fulfill/status" config-
ref="HTTP_Listener_Config"/>
  <step-functions:describe-execution config-
ref="StepFunctions_Config" doc:name="Check execution status"
  executionArn="#[attributes.queryParams.executionArn]"
target="execStatus"/>
  <ee:transform doc:name="Simplify for UI">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{
  status: vars.execStatus.status,
  output: if (vars.execStatus.status == 'SUCCEEDED')
read(vars.execStatus.output, "application/json") else null
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

status values: RUNNING, SUCCEEDED, FAILED, TIMED_OUT, ABORTED

Step Functions reports one of these five states for any execution, so a UI checking status only ever needs to branch on one of a small, fixed set of values rather than parsing free-form state.

`output only parsed when status == 'SUCCEEDED'`

The `output` field is only populated once an execution has actually finished successfully, so guarding the parse behind a status check avoids trying to read a null or empty output for an execution that's still running or that failed.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an `HTTP Listener` onto the canvas and set `Path` to `/orders/fulfill/status`.
- 2 Drag the Step Functions `Describe Execution` operation, bind `Execution Arn` to the query parameter, and set `Target Variable`.
- 3 Drag a `Transform Message` to build the simplified status response, then save and run.

TESTING IN POSTMAN

GET

`https://localhost:8081/orders/fulfill/status?executionArn=arn:aws:states:...`

SAMPLE RESPONSE



JSON

```
{ "status": "SUCCEEDED", "output": { "inventoryReserved": true,
"paymentCharged": true } }
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts an execution ARN, the Step Functions Connector's Describe Execution operation retrieves the current status, and a transform simplifies the response into a small, UI-friendly shape that only parses the actual output once the execution has genuinely succeeded — letting a status page poll independently of whatever originally started the workflow.

#57 Passing Mule Flow Variables into a Step Functions Input Payload

AWS

PROBLEM STATEMENT

The fulfillment state machine needs more context than the raw checkout request contains — a fraud score computed earlier in the flow, a warehouse assignment decided by business logic upstream — and that enriched context has to travel with the execution's input.

DETAILED EXPLANATION

Step Functions has no visibility into a Mule flow's internal variables — the only information any state in the machine ever has access to is whatever was included in the execution's original input, plus whatever earlier states pass forward. Building that input payload deliberately, pulling together everything from earlier in the Mule flow that later states will actually need, is what avoids a state machine having to "call back" into Mule mid-execution just to look something up it could have been given upfront.

CONNECTORS & COMPONENTS USED

AWS

AWS Step Functions Connector – Start Execution

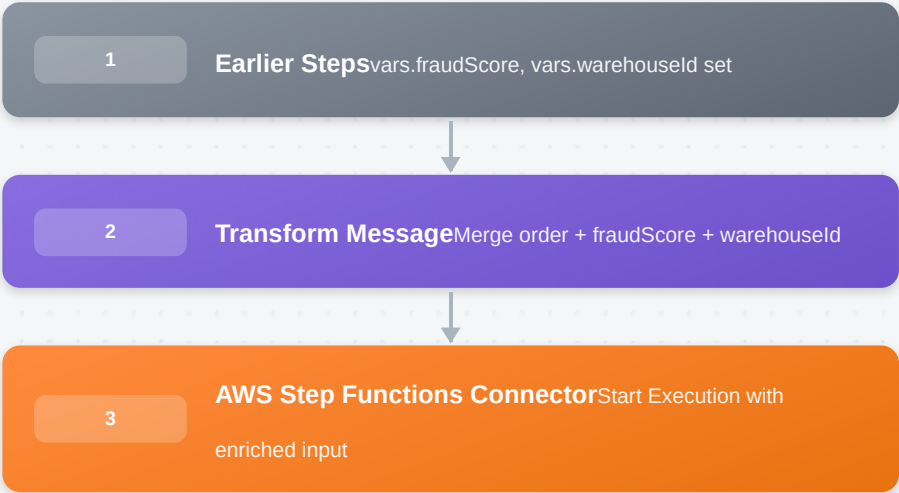
DW

Transform Message

STEPS TO CREATE THE MULE FLOW

- 1 Compute the fraud score and warehouse assignment earlier in the flow, storing each in a variable.
- 2 Assemble those variables together with the original order into one enriched input object.
- 3 Pass that enriched object as the execution's input.

START-ENRICHED-FULFILLMENT-FLOW



MULE CONFIGURATION XML

```

<flow name="start-enriched-fulfillment-flow">
  <ee:transform doc:name="Build enriched input">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
{
  order: payload,
  fraudScore: vars.fraudScore,
  warehouseId: vars.warehouseId,
  requestedAt: now()
}]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <step-functions:start-execution config-ref="StepFunctions_Config"
doc:name="Start with enriched input"
  stateMachineArn="arn:aws:states:us-east-
1:123456789012:stateMachine:order-fulfillment"
  name="#['fulfillment-' ++ vars.orderId]">
    <step-functions:input>#[write(payload, "application/json")]
  </step-functions:input>
  </step-functions:start-execution>
</flow>

```

EXPLANATION OF THE CODE

order: payload nested inside the enriched object

Keeping the original order data nested under its own key, rather than flattening everything into one object, keeps the input's structure self-documenting — a state later in the machine reading `$.order.items` versus `$.fraudScore` makes clear which data came from where.

the enrichment transform runs before start-execution, not inside it

Building the full enriched payload as its own explicit step, rather than inlining a complex expression directly into `step-functions:input`, keeps the mapping logic readable and separately testable from the actual AWS call.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Ensure `fraudScore` and `warehouseId` are already set as flow variables earlier in the flow.
- 2 Drag a `Transform Message` to merge the order payload with those variables into one object.
- 3 Drag the Step Functions `Start Execution` operation and bind `Input` to the merged payload.
- 4 Save and run, then inspect the execution's input in the AWS Console to confirm all three pieces of data arrived.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/fulfill

Header

Value

Content-Type

application/json

EXECUTION INPUT (VISIBLE IN AWS CONSOLE)

JSON

```
{
  "order": { "orderId": "ORD-991", "items": [...] },
  "fraudScore": 12,
  "warehouseId": "WH-EAST-04",
  "requestedAt": "2026-08-28T09:35:00Z"
}
```

BRIEFING ALL THE STEPS IN WORDS

A transform combines the original order payload with earlier-computed fraud score and warehouse assignment variables into one deliberately structured input object, and the Step Functions Connector's Start Execution operation passes it as the execution's input — giving every state in the workflow everything it needs upfront, since nothing in Step Functions can reach back into Mule's flow variables once the execution has started.

#58

Handling a Step Functions Execution Failure and Triggering Compensation

AWS

PROBLEM STATEMENT

When a fulfillment execution fails partway through — inventory was reserved but payment failed — the reserved inventory needs to be released automatically, not left held indefinitely against a payment that will never complete.

DETAILED EXPLANATION

A multi-step process that fails partway through leaves partial side effects behind unless something explicitly undoes them — this is the same compensating-transaction problem any distributed workflow faces. Mule's role here is to watch for a **FAILED** execution status (via Problem #56's Describe Execution, run on a schedule or triggered by a Step Functions failure notification) and, upon detecting one, run a compensation flow that inspects how far the execution got and reverses whatever partial work needs reversing.

CONNECTORS & COMPONENTS USED

CORE Scheduler

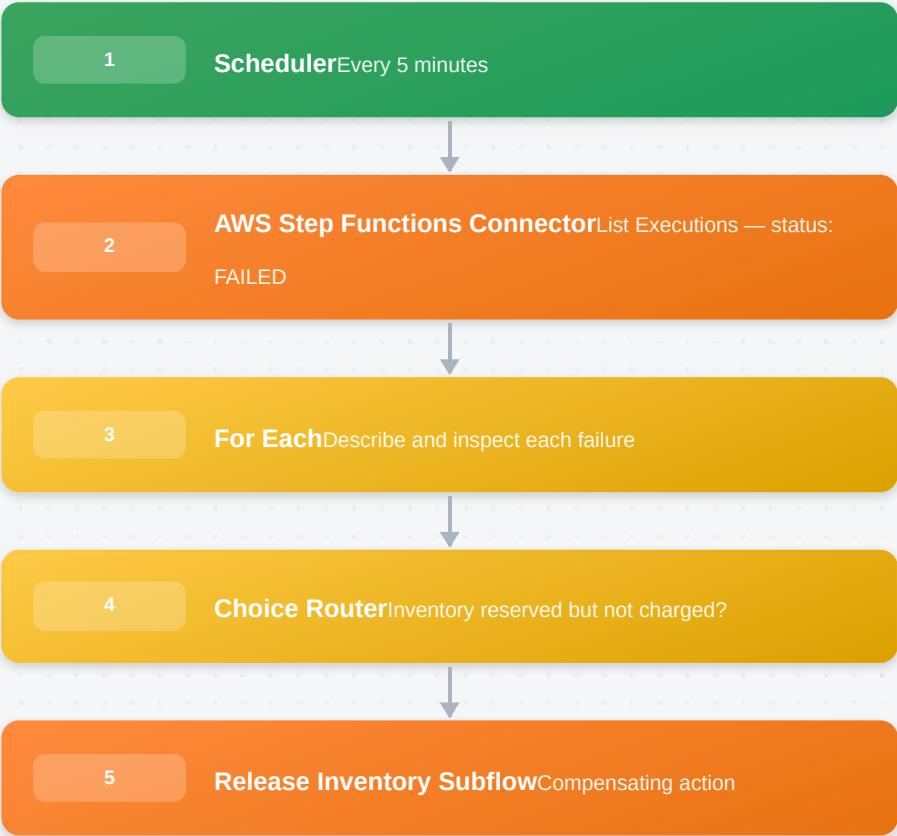
AWS AWS Step Functions Connector – List Executions, Describe Execution

CORE Choice Router

STEPS TO CREATE THE MULE FLOW

- 1 Run periodically, listing recent executions of the fulfillment state machine with status **FAILED**.
- 2 For each failed execution, describe it to see how far it got.
- 3 If inventory was reserved but payment wasn't charged, release the inventory.

COMPENSATE-FAILED-FULFILLMENT-FLOW



MULE CONFIGURATION XML


```

<flow name="compensate-failed-fulfillment-flow">
  <scheduler doc:name="Every 5 minutes">
    <scheduling-strategy>
      <fixed-frequency frequency="300000"/>
    </scheduling-strategy>
  </scheduler>
  <step-functions:list-executions config-ref="StepFunctions_Config"
doc:name="List failed executions"
    stateMachineArn="arn:aws:states:us-east-
1:123456789012:stateMachine:order-fulfillment"
    statusFilter="FAILED" target="failedExecutions"/>
  <foreach collection="#[vars.failedExecutions.executions]">
    <step-functions:describe-execution config-
ref="StepFunctions_Config" doc:name="Inspect failure detail"
    executionArn="#[payload.executionArn]" target="execDetail"/>
    <choice>
      <when expression="#[read(vars.execDetail.input,
"application/json").inventoryReserved and not
(vars.execDetail.output default '' contains 'paymentCharged')]">
        <flow-ref name="release-inventory-subflow"/>
      </when>
    </choice>
  </foreach>
</flow>

```

EXPLANATION OF THE CODE

statusFilter="FAILED" on List Executions

Filtering at the API level to only failed executions means this compensation flow never has to fetch or examine successful executions at all — the AWS call itself does the narrowing.

checking whether output contains 'paymentCharged'

Because a Step Functions execution's output reflects how far it actually got before failing, inspecting it for evidence of the

payment step having run is what distinguishes "inventory reserved, payment never attempted" (needs compensation) from "inventory reserved and paid, something else failed later" (doesn't).

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Scheduler** onto the canvas and set it to run every 5 minutes.
- 2 Drag the Step Functions **List Executions** operation, set **State Machine Arn** and **Status Filter** to **FAILED**.
- 3 Drag a **For Each** scope, then inside it the Step Functions **Describe Execution** operation.
- 4 Drag a **Choice** router checking whether compensation is needed, with a **Flow Reference** to the release-inventory subflow, then save and run.

TESTING IN POSTMAN

Scheduler-triggered. Force a fulfillment execution to fail after inventory reservation but before payment, then expose an admin endpoint to run compensation on demand.

POST `https://localhost:8081/admin/compensate-failures/run-now`

CONSOLE OUTPUT



LOG

WARN Releasing inventory for failed execution fulfillment-ORD-994 (payment never charged)

BRIEFING ALL THE STEPS IN WORDS

A Scheduler periodically lists failed fulfillment executions, describes each one to see exactly how far it got, and a Choice router identifies the specific case of inventory reserved but payment never charged — triggering a compensating release-inventory subflow only for that case, so a partially-failed workflow never leaves stock held indefinitely against an order that's never going to complete.

Combining Lambda and DynamoDB in One Mule-Orchestrated Workflow

AWS

PROBLEM STATEMENT

A product-recommendation endpoint needs to call a Lambda function for the actual recommendation logic, then persist the result to DynamoDB for fast retrieval on subsequent requests — caching an expensive computation behind a fast lookup.

DETAILED EXPLANATION

This is a straightforward but common combination: use Lambda for compute that's genuinely expensive or logic-heavy, and DynamoDB for the fast lookup path once that compute has already run once. Checking DynamoDB first, before ever invoking the Lambda, is what actually delivers the caching benefit — calling the Lambda unconditionally and just also writing to DynamoDB would still pay the full computation cost on every single request.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

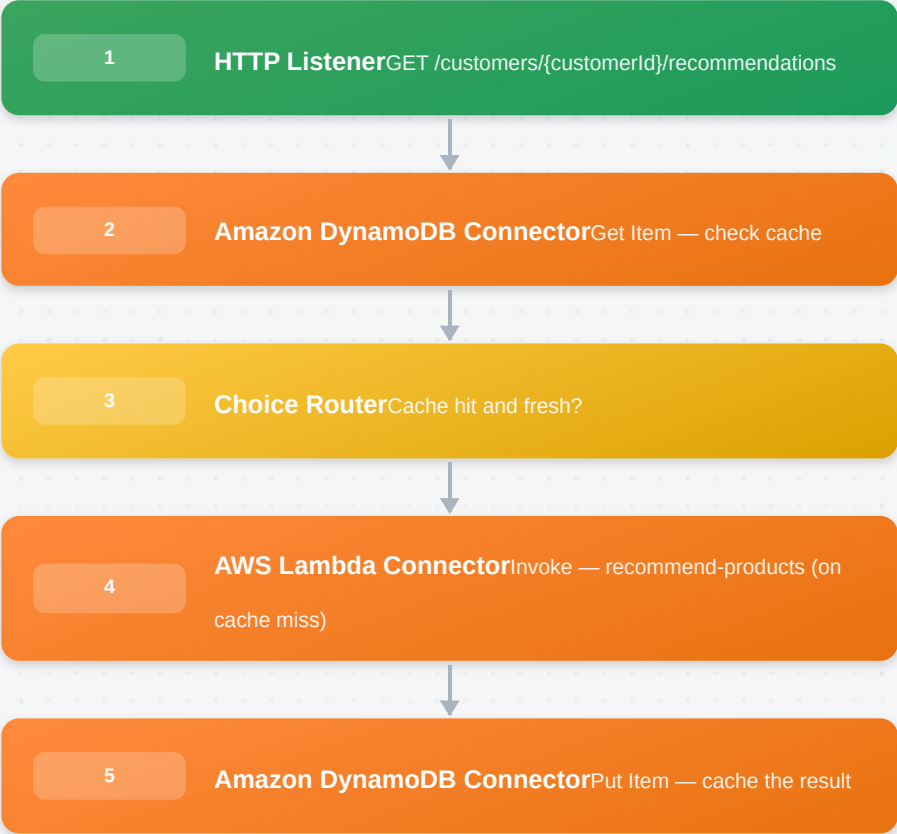
AWS Amazon DynamoDB Connector – Get Item, Put Item

AWS AWS Lambda Connector – Invoke

STEPS TO CREATE THE MULE FLOW

- 1 Accept a customer ID and check DynamoDB for a cached recommendation.
- 2 If found and not stale, return it directly.
- 3 Otherwise, invoke the recommendation Lambda, cache the result in DynamoDB, and return it.

GET-CACHED-RECOMMENDATION-FLOW



MULE CONFIGURATION XML

```

<flow name="get-cached-recommendation-flow">
  <http:listener path="/customers/{customerId}/recommendations"
config-ref="HTTP_Listener_config"/>
  <dynamodb:get-item config-ref="DynamoDB_Config" doc:name="Check
recommendation cache"
    tableName="recommendation-cache" target="cacheResult">
    <dynamodb:key>#{[ { customerId: { S:
attributes.uriParams.customerId } } ]}</dynamodb:key>
  </dynamodb:get-item>
  <choice>
    <when expression="#[not isEmpty(vars.cacheResult.item) and
(now() - (vars.cacheResult.item.cachedAt.S as DateTime)) <<
|PT1H|]">
      <set-payload value="#
[read(vars.cacheResult.item.recommendations.S,
"application/json")]"/>
    </when>
    <otherwise>
      <lambda:invoke config-ref="Lambda_Config" doc:name="Compute
fresh recommendations"
        functionName="recommend-products"
        invocationType="RequestResponse" target="recResult">
        <lambda:payload>#{[ { customerId:
attributes.uriParams.customerId } ]}</lambda:payload>
      </lambda:invoke>
      <dynamodb:put-item config-ref="DynamoDB_Config"
doc:name="Cache fresh result" tableName="recommendation-cache">
        <dynamodb:item>#{[ {
        customerId: { S: attributes.uriParams.customerId },
        recommendations: { S: vars.recResult },
        cachedAt: { S: now() as String }
      } ]}</dynamodb:item>
      </dynamodb:put-item>
      <set-payload value="#[read(vars.recResult,
"application/json")]"/>
    </otherwise>
  </choice>
</flow>

```

```
</choice>
</flow>
```

EXPLANATION OF THE CODE

```
(now() - cachedAt) << |PT1H|
```

Checking the cache entry's age against a one-hour freshness window is what keeps this a genuine cache rather than a permanent, potentially stale snapshot — recommendations older than an hour are treated as a miss and recomputed.

```
DynamoDB Get Item checked before Lambda Invoke, in that order
```

The entire performance benefit of this pattern depends on this ordering — DynamoDB's single-digit-millisecond read happens first, and the comparatively expensive Lambda call is reached only on an actual cache miss.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** onto the canvas and set **Path** to `/customers/{customerId}/recommendations`.
- 2 Drag the DynamoDB **Get Item** operation to check the cache first.
- 3 Drag a **Choice** router checking for a fresh cache hit; in the **when** branch, return the cached value directly.
- 4 In the **otherwise** branch, drag the Lambda **Invoke** operation followed by DynamoDB **Put Item** to cache the fresh result, then save and run.

TESTING IN POSTMAN

GET

https://localhost:8081/customers/CUST-5521/recommendations

SAMPLE RESPONSE (CACHE MISS, COMPUTED FRESH)

JSON

```
[{ "sku": "WIDGET-02", "score": 0.91 }, { "sku": "GADGET-07", "score": 0.84 }]
```

BRIEFING ALL THE STEPS IN WORDS

The HTTP Listener accepts a customer ID, the DynamoDB Connector's Get Item operation checks for a fresh cached recommendation first, and only on a genuine cache miss does the Lambda Connector's Invoke operation compute new recommendations — which are then written back to DynamoDB via Put Item so the next request within the freshness window skips the expensive computation entirely.

#60

Building a Real-Time Order-Enrichment Pipeline Using Kinesis, Lambda, and Mule

AWS

PROBLEM STATEMENT

Closing out Volume 3: raw order events flowing through a Kinesis stream need to be enriched in real time with a customer's lifetime value and risk score — both computed by a Lambda function — before being written to DynamoDB for a live operations dashboard.

DETAILED EXPLANATION

This capstone chains together the volume's core building blocks into a single streaming enrichment pipeline: a Kinesis consumer (#51, #53, #54) reads raw order events as they arrive, a synchronous Lambda invocation (#41, #43) enriches each one with computed customer context, and a DynamoDB write (#45) lands the enriched record where the operations dashboard reads it — the same enrich-then-store shape as Problem #59's cache pattern, but running continuously against a live stream instead of on-demand against individual HTTP requests.

CONNECTORS & COMPONENTS USED

AWS Amazon Kinesis Connector – On New Record

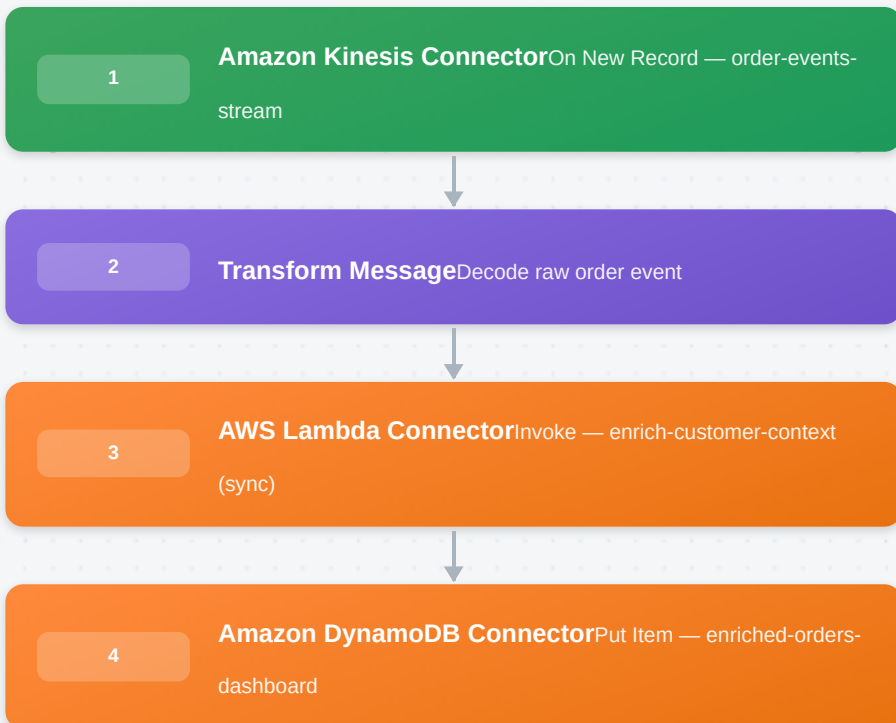
AWS AWS Lambda Connector – Invoke

AWS Amazon DynamoDB Connector – Put Item

STEPS TO CREATE THE MULE FLOW

- 1 Consume raw order events from the order-events-stream Kinesis stream.
- 2 For each event, invoke a Lambda that computes lifetime value and risk score for that customer.
- 3 Write the enriched record to a DynamoDB table the operations dashboard reads from.

ENRICH-ORDER-EVENTS-FLOW



MULE CONFIGURATION XML


```

<flow name="enrich-order-events-flow">
  <kinesis:listener config-ref="Kinesis_Config" doc:name="Watch
order events stream"
    streamName="order-events-stream" shardIteratorType="LATEST"/>
  <ee:transform doc:name="Decode order event">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
---
read(payload.data as String {encoding: "base64"},
"application/json")]</ee:set-payload>
    </ee:message>
  </ee:transform>
  <lambda:invoke config-ref="Lambda_Config" doc:name="Enrich with
customer context"
    functionName="enrich-customer-context"
invocationType="RequestResponse" target="enrichment">
    <lambda:payload>#[{ customerId: payload.customerId }]
  </lambda:payload>
  </lambda:invoke>
  <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Write
enriched record"
    tableName="enriched-orders-dashboard">
    <dynamodb:item>#[%dw 2.0
output application/json
var context = read(vars.enrichment, "application/json")
---
{
  orderId: { S: payload.orderId },
  customerId: { S: payload.customerId },
  orderTotal: { N: payload.total as String },
  lifetimeValue: { N: context.lifetimeValue as String },
  riskScore: { N: context.riskScore as String },
  enrichedAt: { S: now() as String }
}]</dynamodb:item>

```

```
</dynamodb:put-item>  
</flow>
```

EXPLANATION OF THE CODE

a single Kinesis event driving one synchronous Lambda call each

Because the Lambda invocation is synchronous and happens once per Kinesis record, this pipeline's overall throughput is bounded by the Lambda's own latency — a real production version would likely batch records (Problem #54) and enrich several per Lambda call to keep pace with a high-volume stream.

context read from `vars.enrichment` inside the `DynamoDB` item mapping

Parsing the Lambda's JSON string response inline, directly inside the item-construction expression, keeps the enrichment result's

lifetimeValue
and riskScore
fields
available
exactly where
they're needed
to build the
final
DynamoDB
item, without
an extra
intermediate
transform
step.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the Kinesis **On New Record** source onto the canvas, set **Stream Name** to **order-events-stream**.
- 2 Drag a **Transform Message** to decode the raw order event.
- 3 Drag the AWS Lambda **Invoke** operation, set **Function Name** to **enrich-customer-context**, and bind **Payload**.
- 4 Drag the DynamoDB **Put Item** operation and write the combined item mapping, then save and run.

TESTING IN POSTMAN

Stream-triggered. Publish a raw order event to order-events-stream and confirm the enriched record appears in DynamoDB moments later.

POST

```
(AWS CLI) aws kineses put-record --stream-name order-events-stream ...
```

RESULTING DYNAMODB ITEM (VISIBLE IN AWS CONSOLE)

JSON

```
{
  "orderId": "ORD-991",
  "customerId": "CUST-5521",
  "orderTotal": 216.41,
  "lifetimeValue": 4820.00,
  "riskScore": 12,
  "enrichedAt": "2026-08-28T09:40:11Z"
}
```

BRIEFING ALL THE STEPS IN WORDS

The Kinesis Connector's On New Record source reads raw order events from the stream in real time, the Lambda Connector's Invoke operation enriches each one with a synchronously-computed lifetime value and risk score, and the DynamoDB Connector's Put Item operation lands the combined record where the operations dashboard reads it — closing Volume 3 by chaining streaming ingestion, serverless compute, and fast structured storage into one continuously-running enrichment pipeline.

End of Volume 3

Volume 3 moved into compute and structured data: invoking Lambda synchronously and asynchronously with proper error handling, reading and writing DynamoDB with conditional updates and batch operations, consuming and publishing Kinesis Data Streams in real time, and orchestrating multi-step workflows with Step Functions — including detecting and compensating for partial failures.

Volume 4 turns to security, resilience, and observability: IAM and STS authentication, Secrets Manager and KMS, circuit breakers and retries, and CloudWatch logging, metrics, and tracing — the patterns that keep a MuleSoft–AWS integration safe and debuggable in production — Problems 61–80.